

WHITEPAPER · VERSION 0.2 · DRAFT

# Code that *cannot* be seen.

*A programming language where privacy is not a feature — it is the foundation.*

Today, every line of source code that runs in the world exists, at some point, as readable plaintext. This is the original sin of software security. **Arcanum** is a compiler layer proposal where source code privacy is treated as a first principle — combining Trusted Execution Environments (TEE) for immediate, practical protection with Zero-Knowledge proofs as the long-term cryptographic foundation. Developers write in the language they already know. Arcanum handles the rest. Plaintext source never exists post-compilation. Not on the server. Not in memory. Not anywhere a hacker, a competitor, or a state actor could find it.

AUTHOR

Mert Atakan

STATUS

Conceptual Draft

VERSION

0.1 — Pre-MVP

YEAR

2026

CATEGORY

TEE + ZK / Compiler Security

## 01 The Problem

Software today has a fundamental and largely ignored vulnerability: **source code is readable**. Even when compiled into machine instructions, determined attackers with enough access — whether through a breach, an insider threat, or supply-chain compromise — can eventually reconstruct what a program does and how it works.

Existing countermeasures treat this as a problem of access control: firewalls, encryption at rest, obfuscation techniques. But these are all variations of the same idea — hide the key, not the lock. If the key is found, or the right person is bribed, the code is exposed.

*"The most secure secret is one that cannot be read — not one that is merely difficult to find."*

The consequences are tangible: billions of dollars lost to code theft, competitive advantage eroded by reverse engineering, critical infrastructure made vulnerable by the assumption that compiled binaries are "safe." The world does not need better locks. It needs a different kind of vault.

secret itself — suggest a radical alternative. What if the same principle could be applied not just to data, but to **code itself**?

## 02 The Solution

**Arcanum** is not a new programming language. It is a **privacy-first compiler layer** that sits on top of languages developers already use — C++, Rust, JavaScript, TypeScript, Go, and any other language supported by LLVM or RISC-V toolchains. Developers write code in the language they know. Arcanum transforms it.

The single, radical design constraint is this: **source code must never exist as readable plaintext after the compilation step**. Arcanum achieves this through a two-phase approach — practical protection today, with a path to stronger mathematical guarantees tomorrow.

**Phase 1 — TEE (Trusted Execution Environment):** In the near term, Arcanum wraps compiled code inside hardware-enforced secure enclaves (Intel SGX, AMD SEV, ARM TrustZone). The code enters the enclave encrypted, executes inside it, and

logic. This is how the world's most security-conscious companies already protect their most sensitive algorithms — Arcanum makes it automatic and developer-transparent.

**Phase 2 — ZK (Zero-Knowledge Proofs):** As ZK compiler infrastructure matures, Arcanum will transition to transforming source code into zero-knowledge arithmetic circuits — a mathematical representation that is provably unreadable without the original source, requires no trusted hardware, and adds post-quantum resistance. The long-term vision is a system where source code privacy is guaranteed by mathematics alone, not by trust in any hardware manufacturer.

#### HOW ARCANUM WORKS – CONCEPTUAL FLOW

```
// Step 1: Developer writes in any supported language (e.g. Rust)
```

```
fn transfer_funds(sender: &Account, receiver: &Account,  
amount: u64) {  
    assert!(sender.balance >= amount);  
    sender.balance -= amount;  
    receiver.balance += amount;  
}
```

```
// Step 2: Arcanum compiler layer – developer runs one command
```

```
// $ arcanum compile transfer.rs --output transfer.enc  
//
```

```
// Phase 1 output: TEE-wrapped encrypted binary  
// → Executes inside a hardware secure enclave  
// → Source logic unreadable even to server owner  
// → No TEE or ZK knowledge required from the developer  
//
```

```
// Phase 2 output (future): ZK arithmetic circuit  
// → Mathematically unreadable without original source
```

This is not obfuscation. Obfuscation makes code hard to read; a determined reverse engineer can still decode it. TEE enclaves enforce hardware-level isolation that cannot be bypassed through software. And ZK circuits are mathematically impossible to reverse without the original source — no amount of compute changes that.

In both phases, the developer experience is identical: write code as usual, run one command, ship a protected binary. The complexity is entirely Arcanum's responsibility.

## 03 Technical Architecture

Arcanum's architecture is designed in two phases — a practical near-term implementation using proven hardware security, and a long-term cryptographic foundation using zero-knowledge proofs. Both phases share the same developer-facing interface: one compile command, complete source code privacy.

## Layer 1 — The Compiler Layer

Accepts source code in any supported language (C++, Rust, JavaScript, TypeScript, Go, and others via LLVM or RISC-V toolchains). In Phase 1, produces a TEE-wrapped encrypted binary. In Phase 2, produces a STARK-compatible arithmetic circuit. The compilation step is the only moment source code exists in plaintext.



## Layer 2A — TEE Enclave (Phase 1)

The compiled binary executes inside a hardware-enforced secure enclave (Intel SGX, AMD SEV, or ARM TrustZone). The source logic is invisible to the operating system, the server owner, and any external attacker. Performance overhead is minimal — near-native execution speed. Trust is placed in the hardware manufacturer, not in any software layer.



## Layer 2B — ZK Proof System (Phase 2)

Generates zero-knowledge proofs (STARK-based) for each execution. No trusted hardware required. Proofs are compact, fast to verify, and reveal nothing about internal program state. Post-quantum resistant by design. This phase eliminates the hardware trust assumption entirely — privacy is guaranteed by mathematics alone.



### Layer 3 — The Verifier

A lightweight verification layer that any third party can use to confirm that a program ran correctly — without accessing the source logic. In Phase 1, verification is attestation-based (TEE remote attestation). In Phase 2, verification is cryptographic proof-based. The public-facing result is identical: execution confirmed, logic remains private.

**Why TEE first?** Trusted Execution Environments are battle-tested, widely deployed (AWS Nitro, Azure Confidential Computing, Google Confidential VMs), and impose near-zero performance penalty. They provide immediate, real-world protection for proprietary source code today — while the ZK compiler infrastructure needed for Phase 2 continues to mature across the research community.

**Why ZK eventually?** TEE places trust in hardware manufacturers. If Intel or AMD has a vulnerability — or is compelled by a government — the enclave can theoretically be compromised. ZK circuits have no such dependency: their privacy guarantee is mathematical, not institutional. Phase 2 is not a replacement for Phase 1 — it is an upgrade path that removes the final trust assumption.

04

Honest Threat Model

No system is unconditionally secure. The value of Arcanum is not the claim of absolute protection — it is a significant, measurable reduction in the attack surface for source code theft and reverse engineering. The following table reflects an honest assessment of where the system succeeds and where residual risks remain.

| THREAT                                 | STATUS       | NOTES                                                                                                                                                                           |
|----------------------------------------|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Source code theft via network breach   | MITIGATED    | Plaintext source never transmitted or stored post-compile. Both TEE and ZK phases enforce this.                                                                                 |
| Reverse engineering of compiled output | MITIGATED    | Phase 1: TEE enclave prevents access to decrypted logic at runtime. Phase 2: ZK circuits carry no recoverable linguistic information.                                           |
| Hardware manufacturer compromise (TEE) | PHASE 1 ONLY | Phase 1 trusts Intel/AMD/ARM. A hardware-level vulnerability or government compulsion could theoretically expose enclave contents. Phase 2 eliminates this dependency entirely. |



Insider threat  
 (developer leaks  
 source)

UNMITIGATED

Social and organizational  
 controls required. Out of scope  
 for the compiler layer in both  
 phases.

Compiler-level  
 vulnerabilities

PARTIAL

A bug in the compiler could  
 expose intermediate  
 representations. Formal  
 verification of the compiler is a  
 long-term priority.

Side-channel  
 attacks

PARTIAL

Phase 1: TEE hardware provides  
 some side-channel mitigations.  
 Phase 2: ZK circuits do not  
 inherently protect against timing  
 or power analysis at the  
 hardware level.

Quantum  
 computing  
 attacks

MITIGATED  
 (Phase 2)

Phase 1: TEE is not affected by  
 quantum attacks on  
 cryptography. Phase 2: STARK-  
 based systems are post-  
 quantum resistant by design.

05

## Existing Landscape

Several mature efforts exist at the intersection of zero-knowledge cryptography and compiler design. Arcanum builds on the same infrastructure as these projects — but addresses a fundamentally different problem.

**zkLLVM** by NilFoundation is an LLVM-based circuit compiler that supports C++, Rust, JavaScript, and TypeScript. It transforms existing code into ZK circuits efficiently. However, its primary goal is *verifiable computation* — proving that a program ran correctly — not source code privacy. The developer's source remains readable; the circuit is a proof of execution, not a replacement for the code.

**RISC Zero** is a zkVM that generates ZK proofs for Rust and C++ programs compiled to RISC-V. Like zkLLVM, it targets execution verifiability. The source code is never hidden — it is simply executed inside a provable environment.

**SP1** by Succinct follows the same pattern: standard Rust code, verifiable execution, no source code privacy guarantee.

**Circom** requires developers to write directly in circuit language — it is a tool for ZK experts, not a transparent layer for mainstream developers.

*The distinction is precise: existing tools ask "can we prove this code ran correctly?" Arcanum asks "can we make this code permanently unreadable – while still provably executable?" These are different questions with different architectures, different threat models, and different target users.*

Arcanum uses the same underlying compiler infrastructure as zkLLVM and RISC Zero — LLVM and RISC-V toolchains — but reorients the design goal entirely around **source code privacy as a first principle**. The developer experience is also different: no ZK background required, no circuit writing, no annotations beyond a standard compile command.

No production-ready tool with this specific design constraint currently exists. This is a genuine and addressable gap, at the intersection of growing enterprise demand for intellectual property protection and maturing ZK compiler infrastructure.

# Roadmap

01

CURRENT — PHASE 0

## Conceptual Foundation

Defining the core architecture, threat model, and two-phase roadmap. Publishing this whitepaper to initiate community feedback and identify technical collaborators with TEE and/or ZK compiler experience.

02

PHASE 1 — TEE MVP

## TEE-Based Compiler Layer

A working Arcanum compiler layer that wraps Rust or C++ programs inside TEE enclaves (targeting Intel SGX / AWS Nitro). Target: one end-to-end working example — developer writes code, runs one command, receives a TEE-protected binary. Source logic unreadable at runtime even with root access.

03

PHASE 2 — MULTI-LANGUAGE &amp; TOOLING

## Language Expansion & Developer Experience

Extend TEE compiler support to the full range of LLVM and RISC-V compatible languages. IDE integrations, debugging tools, and documentation targeting developers with no TEE background. First enterprise pilots.

**Zero-Knowledge Compiler Layer**

Introduce the ZK compiler backend alongside the TEE layer. Developers can opt into ZK output for use cases requiring mathematical privacy guarantees without hardware trust. STARK-based, post-quantum resistant. Collaboration with ZK researchers to validate cryptographic security claims.

05

## PHASE 4 — AUDITED RELEASE

**Independent Security Audit & Public Beta**

Independent audit of both the TEE and ZK compiler layers. Public beta targeting financial applications, proprietary algorithms, and critical infrastructure. Full post-quantum migration path documented.

**Code has always been readable.  
It doesn't have to be.**

The history of cryptography is the history of humanity learning to communicate secrets across hostile environments. Arcanum applies that same principle to software itself — not as a patch, not as a layer, but as the language layer.

This whitepaper is a beginning. The architecture is intentionally open for critique, collaboration, and revision. The goal is not to have all the answers, but to ask the right question clearly:

Feedback, contributions, and technical collaboration are welcome. This document will be updated as the design matures.

#### AUTHORSHIP

Idea & Vision: **Mert Atakan**

Compiled & Written by: **Claude** (Anthropic)

#### SPECIAL THANKS

Atakan and Erdem Families

Cem Buldanlioğlu

#### PUBLICATION

arXiv · GitHub · ethresear.ch

---

Arcanum · Whitepaper v0.2 · 2026 · Draft – All rights reserved